
– RAPPORT ET SOUTENANCE –
SAE2.01
DÉVELOPPEMENT D'UNE
APPLICATION
BRIQU'IUTO



Membre du Groupe:

- **JUSKO Nathaniel** (Chef de Projet)
- DESSENEUX-AURIOL Florient
- MAKHLOUF Zakaria
- BELHAMIDA Adil
- BOUSLIM Mourad



Introduction.....	3
Les ressources mobilisés.....	4
Les apprentissage critiques.....	5
Analyse de la base de données.....	6
A) Explication du MLD/MCD.....	7
B) Insertions et requêtes.....	9
Analyse UML.....	10
A) Analyse Diagramme de classe.....	10
1. Les classes indispensables.....	12
2. Les multiplicités.....	12
Justification de nos choix :.....	13
1. La classe abstraite Boite.....	13
2. L'interface Utilisateur.....	13
4. Le pattern Façade BriquesCollectionManager.....	13
B) Justification diagramme de cas d'utilisation.....	15
Rôle du collectionneur.....	15
Fonctions liées à la personnalisation.....	15
Rôle de l'administrateur.....	15
Relations entre les cas d'utilisation.....	16
C) Présentation du diagramme de séquence.....	16
Diagramme d'activité.....	19
Implementation : explication de la démarche.....	21
A) présenter les maquettes et les choix ergonomique.....	21
Elaboration des Interfaces Hommes Machine.....	27
A) Expliquez la démarche (développement du back-end : traitements, calculs).....	27
Organisation du travail de groupe.....	28
A) Comment vous êtes-vous organisés tout au long de cette SAE ?.....	28
B) Présentez vos outils de gestion de projet (Diagramme de Gantt, matrice RACI).....	29
Explication du Gantt.....	31
C) Présentez GitHub et son utilité et expliquez la manière dont vous l'avez utilisé.....	32
Travail réalisé durant les séances.....	33
Bilan du groupe.....	36
A) Qu'est-ce qui fonctionne, qu'est-ce qui ne fonctionne pas, et pourquoi ?.....	36
B) Bilan sur l'organisation du groupe et du projet.....	37
C) Bilan sur les principaux acquis.....	38

Introduction

Ce rapport présente le travail que nous devons réaliser dans le cadre de la SAÉ 2.01 “Développement d'une application”. L'objectif de ce projet est de créer une application en Java qui se nomme Briqu'IUTO. Elle est destinée à un collectionneur de LEGO qui a besoin d'un outil pour gérer ses boîtes, ses pièces et ses figurines.

Pour répondre aux besoins du client nous avons dû prévoir deux types d'utilisateurs:

Le collectionneur(client) : il peut rechercher des boîtes, voir le détail des pièces et créer ses propres modèles personnalisés.

L'administrateur : il s'occupe de mettre à jour les données, par exemple en ajoutant de nouvelles boîtes ou de nouveaux thèmes dans le système.

Premièrement, nous allons voir en quoi cette SAE, nous apprend les apprentissages critiques du projet et les ressources qui ont été mobilisés. Ensuite une analyse de la base données comportant l'explication du MLD et du MCD et nos choix concernant les insertions. Ceci est suivi d'une analyse de l'UML, des implémentations. Par la suite nous verrons comment sont faites nos interfaces hommes machines. Également, une partie avec les organisations du travail de groupe et le travail réalisé par chacun durant les différentes séances.

Les ressources mobilisés

Durant ce projets, différentes ressources ont été mobilisée :

R2.01 | Développement orienté objets

Pour mener à bien ce projet nous avons utilisé un développement orienté objets afin de faciliter la mise en place du système. A travers cette ressource, nous avons utilisé le langage de programmation Java.

R2.03 | Qualité de développement

Pour procéder à un projet de qualité, la ressource R2.03 a été indispensable pour gérer les "bonnes pratiques" du métier de développeur. Il inclut l'écriture d'un code propre et lisible, la gestion des versions avec les notions abordés : création d'un dépôt distant, synchronisation des dépôts, Méthode TDD, Fichiers JSON pour le travail en équipe (l'utilisation de Git/GitHub), et la mise en place de tests rigoureux pour gérer les cas d'erreurs avec les exceptions en effet la gestion des exceptions se fait en deux parties : une méthode confrontée à une situation rare peut la signaler en levant une exception, ce qui interrompt son exécution ; une méthode qui s'attend à ce que cette situation puisse se présenter au cours de l'exécution d'une partie de son code et qui sait comment y réagir peut rattraper l'exception, que celle-ci soit levée dans une fonctions qu'elle appelle directement ou indirectement..

R2.13 | Communication technique

C'est la capacité à expliquer notre travail informatique. Cette ressource comprend la rédaction de notre rapport final de manière professionnelle, la création de documentations claires, et la préparation de notre soutenance orale.

Les apprentissage critiques

Durant cette SAE, plusieurs apprentissages nous sont fait, améliorer :

AC11.01 | Implémenter des conceptions simples

Cet apprentissage est fait durant tout le projet, par la conception des méthodes, et des implémentations en Java.

AC11.02 | Élaborer des conceptions simples

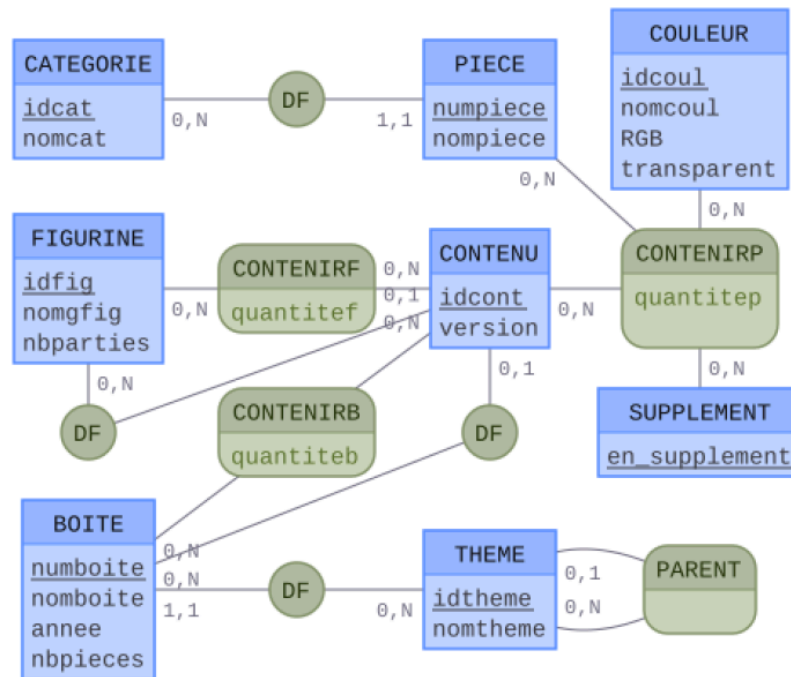
Durant ce projet, différentes conceptions sont faites, du code java aux différents diagrammes et doivent être compris par tous.

AC11.03 | Faire des essais et évaluer leurs résultats en regard des spécifications

Techniquement nous avons utilisé le langage Java et nous nous sommes connectés à une base de données via JDBC. Nous avons aussi appris à organiser notre code avec une architecture propre, à faire des schémas UML pour préparer le développement et à tester notre travail avec JUnit. Pour travailler en équipe, nous avons utilisé l'outil Git et Google Doc. Dans la suite de ce document nous allons d'abord expliquer l'analyse de la base de données et nos diagrammes UML. Ensuite nous détaillerons le code et la création des interfaces. Enfin, nous finirons par un bilan sur notre organisation de groupe et ce que ce projet nous a apporté.

I. Analyse de la base de données

une base de données est **une collection d'informations interdépendantes**



Un MCD (Modèle Conceptuel de Données) est une **représentation schématique qui illustre les données d'une entreprise (ou d'un projet spécifique)**

I.1 Explication du MLD/MCD

Ce modèle représente un système de gestion d'inventaire pour des jeux de construction, comme les LEGO. Il est centré autour de l'entité principale appelée **CONTENU**, qui permet de recenser tous les éléments présents dans une boîte : les pièces (avec leurs couleurs), les figurines, ou encore les sous-boîtes.

Le Modèle Logique de Données (MLD) traduit cette organisation en une structure de tables de base de données. On y retrouve les différentes tables correspondant aux entités, ainsi que les relations entre elles formalisées par des clés.

Par exemple, certaines relations sont de type 1:N (un-à-plusieurs), comme le fait qu'une pièce appartient à une seule catégorie, alors qu'une catégorie peut regrouper plusieurs pièces. Dans le MLD, ces relations se traduisent par l'ajout d'une clé étrangère dans la table "fille" (la pièce) pointant vers la table "mère" (la catégorie).

D'autres relations sont de type N:M (plusieurs-à-plusieurs). C'est le cas pour la répartition des pièces au sein des boîtes. Pour représenter cela, le MLD utilise des tables de liaison (ou tables intermédiaires) qui permettent de lier les entités concernées tout en stockant des informations spécifiques comme la quantité.

clé primaire en **gras**

clé étrangère en *#italique*

CATEGORIE (**idcat**, nomcat)

COULEUR (**idcoul**, nomcoul, RGB, transparent)

SUPPLÉMENT (en_supplement)

THEME (**idtheme**, nomtheme, *#idtheme_pere*)

idtheme_pere fait référence à la clé primaire de la classe **THÈME**

PIECE (**numpiece**, nompiece, *#idcat*)

idcat fait référence à la clé primaire de la classe **CATEGORIE**

FIGURINE (**idfig**, nomgfig, nbparties)

BOITE (**numboite**, nomboite, annee, nbpieces, *#idtheme*)

idtheme fait référence à la clé primaire de la classe **THÈME**

CONTENU (**idcont**, version, *#numboite*, *#idfig*)

numboite fait référence à la clé primaire de la classe **BOITE**

idfig fait référence à la clé primaire de la classe **FIGURINE**

CONTENIRP (*#idcont*, *#numpiece*, *#idcoul*, *#en_supplement*, quantitep)

idcont fait référence à la clé primaire de la classe **CONTENU**

numpiece fait référence à la clé primaire de la classe **PIECE**

idcoul fait référence à la clé primaire de la classe **COULEUR**

en_supplement fait référence à la clé primaire de la classe **SUPPLÉMENT**

CONTENIRF (*#idcont*, *#idfig*, quantitef)

idcont fait référence à la clé primaire de la classe **CONTENU**

idfig fait référence à la clé primaire de la classe **FIGURINE**

CONTENIRB (*#idcont*, *#numboite*, quantiteb)

idcont fait référence à la clé primaire de la classe **CONTENU**

numboite fait référence à la clé primaire de la classe **BOITE**

I.2 Insertions et requêtes

Les insertions sont les données initiales insérées dans la base de données. Ce sont des données récupérées sur le site officiel de recensement des pièces LEGO. Ces insertions fonctionnent de manière à ce que l'on charge les données depuis un fichier CSV, qui contient soit des liens vers des sites de recensement de pièces, soit vers un autre fichier CSV qui renvoie lui aussi vers des sites de recensement de pièces, ou vers un fichier CSV, et ainsi de suite jusqu'à ce que toutes les données soient récupérées et insérées dans les tables. Elles ont pour utilité de tester cette base de données et de nous aider à réaliser toutes les requêtes nécessaires.

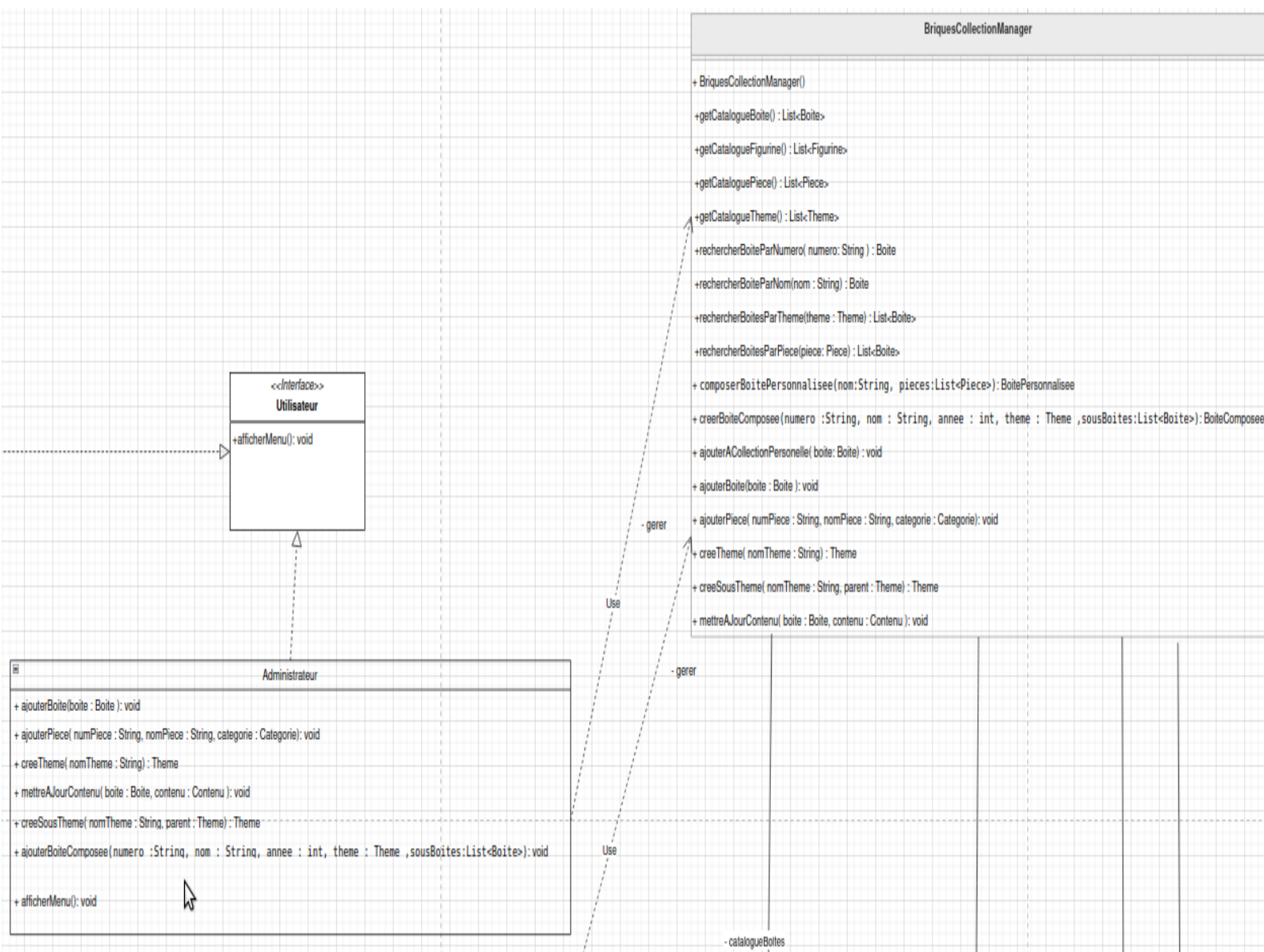
Les requêtes réalisées ont pour objectif de répondre aux différentes demandes de l'utilisateur, afin de chercher différentes informations (comme un catalogue, une pièce, une couleur, une catégorie, etc.). Elles exploitent les insertions. Par exemple, l'une des requêtes a pour but de donner les informations sur les boîtes qui contiennent une pièce de couleur "Very Light Orange", ou encore une autre qui permet de récupérer les informations des pièces qui ne sont utilisées que dans des boîtes du thème "Fortnite".

Ces requêtes vont nous être utiles lors de la création de l'application en Java, car nous allons pouvoir les réutiliser afin de les implémenter dans l'application.

Analyse UML

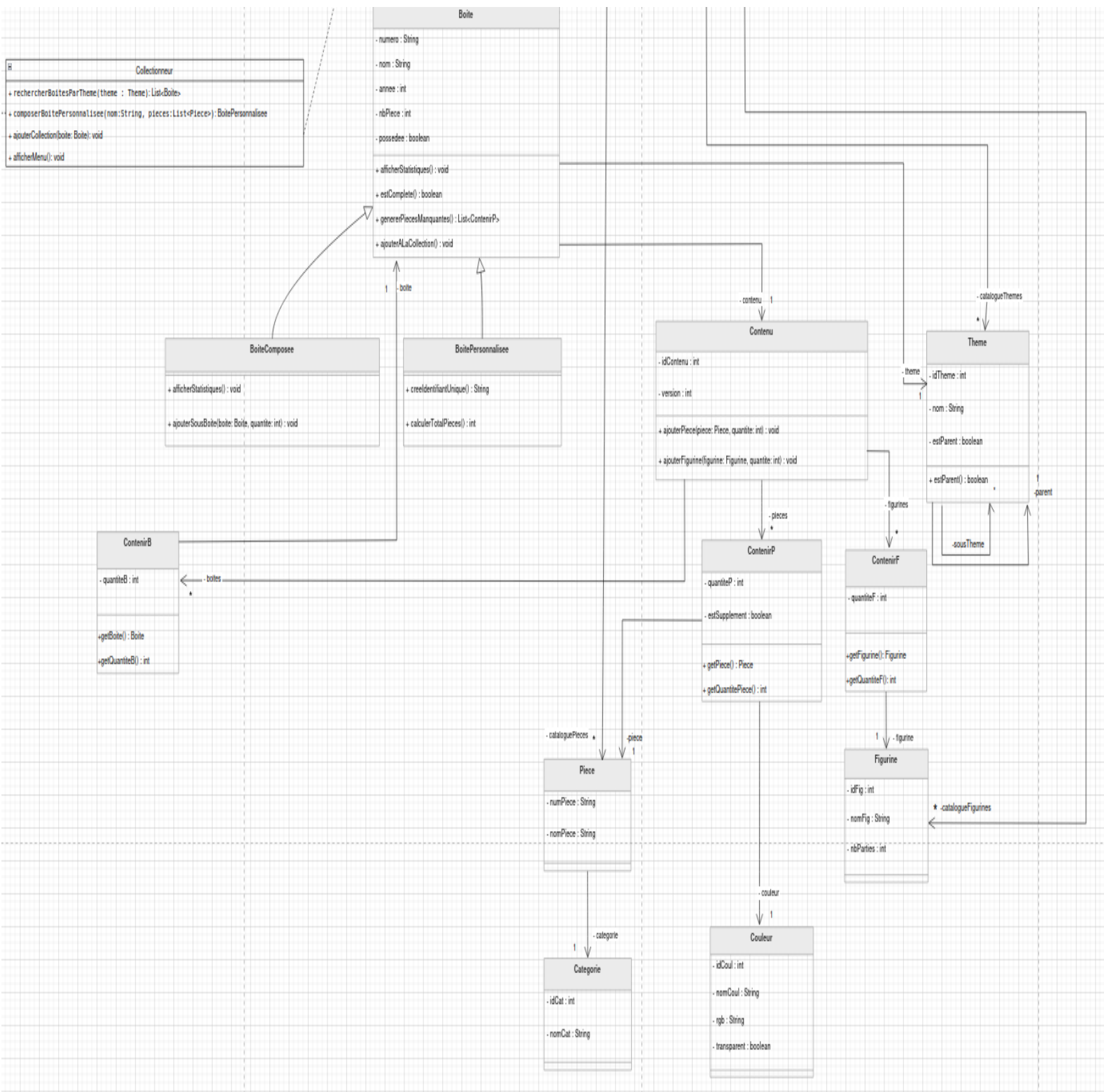
Un diagramme UML est un schéma graphique standardisé qui permet de modéliser visuellement la structure et le comportement d'un logiciel. Il sert de plan universel pour faciliter la communication et la conception entre les développeurs, les chefs de projet et les clients.

A) Analyse Diagramme de classe



Le diagramme de classes exposé ici modélise une application complète de gestion de collections de briques de construction, structurée autour d'un contrôleur central appelé **BriquesCollectionManager** qui orchestre l'ensemble des recherches et des catalogues. À travers une interface **Utilisateur**, le système sépare distinctement les rôles entre l'**Administrateur**, qui possède les droits pour enrichir la base de données globale, et le **Collectionneur**, qui gère son inventaire personnel de modèles officiels

(BoiteComposee) ou de créations propres (BoitePersonnalisee). Ces boîtes, classifiées de manière arborescente par thèmes et sous-thèmes, décrivent leur composition exacte en figurines et en briques individuelles grâce à des classes de liaison (Contenu), permettant d'associer précisément chaque pièce à sa quantité, sa catégorie et ses propriétés graphiques spécifiques telles que son nom de couleur, son code RGB et sa transparence.



1. Les classes indispensables

Le diagramme de classe est constitué de plusieurs classes dont la présence est indispensable au bon fonctionnement du système.

Boite est la classe mère du modèle. Elle centralise les attributs communs à toutes les boîtes (numero, nom, annee, nbPiece, possedee) et sert de superclasse pour l'héritage. Sans elle, impossible d'assurer un traitement polymorphique uniforme des boîtes dans le catalogue..

BriquesCollectionManager est la classe façade du système. Elle agrège les quatre catalogues (List<Boite>, List<Piece>, List<Figurine>, List<Theme>) et expose l'ensemble des opérations disponibles. Elle coordonne les interactions entre les différents catalogues et centralise l'ensemble des opérations exposées aux classes clientes.

Contenu ainsi que ses classes d'association ContenirP, ContenirF et ContenirB sont indispensables pour modéliser le contenu d'une boîte avec précision. Une simple liste ne suffirait pas car il faut stocker la quantité de chaque élément, et dans le cas de ContenirP, le booléen estSupplement pour distinguer les pièces incluses des suppléments.

Theme est essentiel pour la fonctionnalité de recherche par thème exigée par le sujet. Sa relation récursive (un thème peut avoir un thème parent et des sous-thèmes) permet de modéliser une hiérarchie de thèmes sans multiplier les classes.

Pièce et Figurine représentent les éléments de base d'une boîte. Elles sont reliées respectivement à ContenirP et ContenirF et constituent les unités élémentaires que le collectionneur cherche à compléter.

2. Les multiplicités

Les multiplicités présentes dans le diagramme traduisent les règles métier du système.

Boite — Contenu (1 — 1) : chaque boîte possède exactement un contenu qui lui est propre.

Contenu — ContenirP / ContenirF / ContenirB (1 — *) : un contenu regroupe plusieurs lignes d'éléments (pièces, figurines ou sous-boîtes), chacune portant sa quantité.

ContenirP — Piece / ContenirF — Figurine (* — 1) : une même pièce ou figurine peut apparaître dans plusieurs boîtes, mais chaque ligne pointe vers un seul élément.

Boite — Theme (* — 1) : plusieurs boîtes peuvent partager le même thème, mais une boîte n'appartient qu'à un seul thème.

Theme — Theme (0..1 — *) : un thème peut avoir un thème parent et plusieurs sous-thèmes, modélisant ainsi une hiérarchie (ex. : Star Wars est un sous-thème de Cinéma).

BriquesCollectionManager — Boite (1 — *) : le gestionnaire centralise l'ensemble des boîtes du catalogue.

Justification de nos choix :

1. La classe abstraite Boite

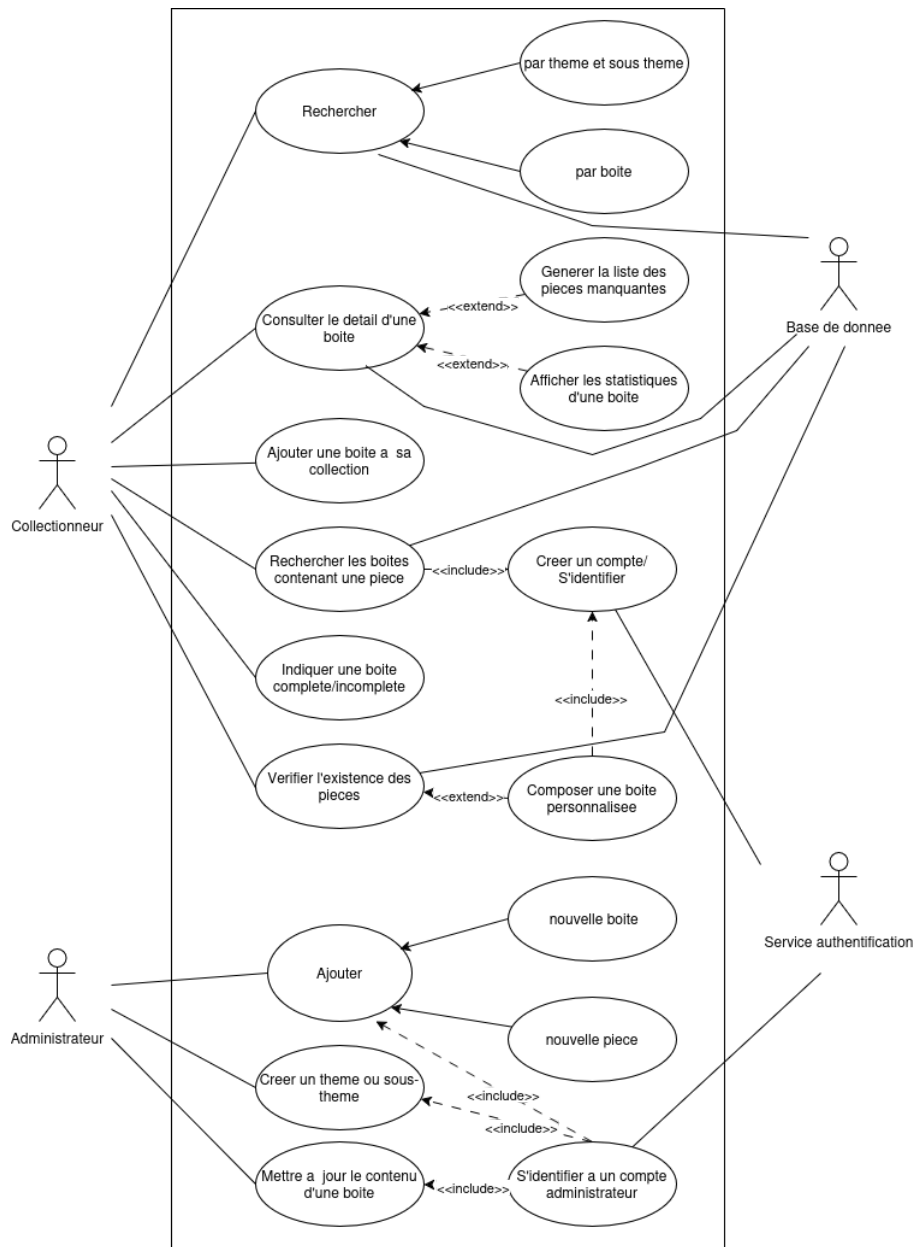
Boite est déclarée abstraite car dans la réalité modélisée, une boîte existe toujours sous une forme précise soit une BoiteComposee, soit une BoitePersonnalisee jamais sous une forme générique. La méthode afficherStatistiques() est également abstraite, car son comportement varie selon le type concret : BoiteComposee devant, en plus, afficher les statistiques de ses sous-boîtes. En revanche, estComplete() et genererPiecesManquantes() sont définies directement dans Boite, leur implémentation étant commune à toutes les sous-classes, en exploitant le polymorphisme.

2. L'interface Utilisateur

Une interface a été préférée à une classe abstraite car Collectionneur et Administrateur ne partagent aucun attribut commun, uniquement un contrat comportemental (afficherMenu()). Ce choix respecte la séparation des responsabilités : les droits d'accès au BriquesCollectionManager diffèrent selon le profil, et l'interface matérialise ce contrat sans imposer de hiérarchie de données inutile.

4. Le pattern Façade BriquesCollectionManager

Le BCM centralise toutes les opérations du système. Les méthodes comme rechercherBoitesParTheme() et composerBoitePersonnalisee() y sont placées car c'est lui qui détient les catalogues. Les classes utilisateurs délèguent systématiquement au BCM, sans manipuler directement les objets métier ce qui garantit un code maintenable et extensible tout en répondant aux exigences de la SAÉ .



B) Justification diagramme de cas d'utilisation

Ce diagramme de cas d'utilisation présente les interactions entre deux acteurs principaux, le Collectionneur et l'Administrateur, dans un système de gestion de boîtes de construction. Il montre les différentes actions possibles selon le rôle de chaque utilisateur, ainsi que les liens entre certains cas d'utilisation.

Rôle du collectionneur

Le collectionneur est l'acteur principal du système. Il peut effectuer plusieurs actions liées à la recherche, à la consultation et à la gestion de ses boîtes. Il peut par exemple rechercher une boîte, consulter le détail d'une boîte, ou encore explorer les boîtes par thème et sous-thème. Il a aussi accès à des fonctions plus avancées comme afficher les statistiques d'une boîte, rechercher les boîtes contenant une pièce, ou ajouter une boîte à sa collection.

Le diagramme montre également qu'il peut indiquer si une boîte est complète ou incomplète. Dans ce cas, le système peut alors générer la liste des pièces manquantes, ce qui permet au collectionneur de mieux suivre l'état de sa collection.

Fonctions liées à la personnalisation

Une autre partie importante du diagramme concerne la composition d'une boîte personnalisée. Pour réaliser cette action, le système doit d'abord vérifier l'existence des pièces. Cette vérification est une étape obligatoire, car elle permet de s'assurer que les éléments nécessaires sont bien disponibles dans la base de données.

Le cas Créer un compte / S'identifier est aussi présent. Il est logique qu'un utilisateur doit posséder un compte pour accéder à certaines fonctionnalités, notamment celles liées à la personnalisation et à la gestion de collection.

Rôle de l'administrateur

L'administrateur a un rôle différent. Il intervient principalement dans la gestion du contenu du système. Il peut ajouter une nouvelle boîte, ajouter une nouvelle pièce, créer un thème ou sous-thème, et mettre à jour le contenu d'une boîte.

Son rôle est donc plus orienté vers la maintenance et l'enrichissement de la base de données. Contrairement au collectionneur, il ne consulte pas seulement les données : il les alimente et les met à jour.

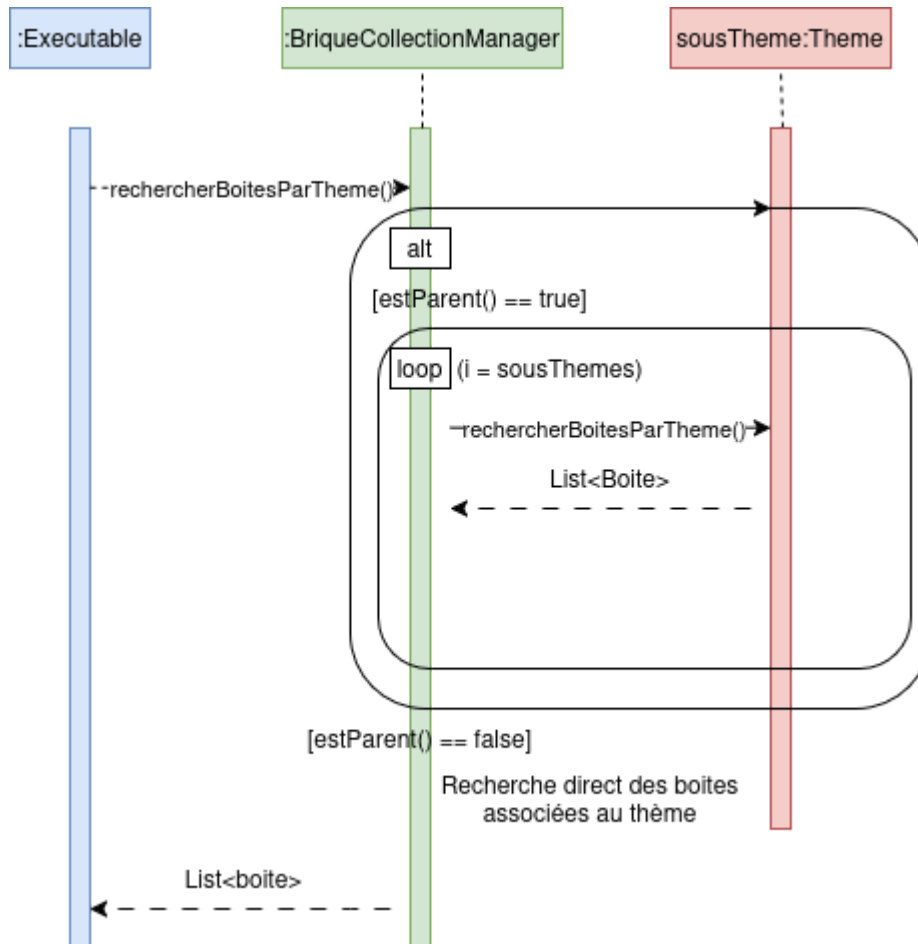
Relations entre les cas d'utilisation

Le diagramme utilise les relations <<include>> et <<extend>>. La relation include indique qu'un cas d'utilisation en appelle obligatoirement un autre. Par exemple, la composition d'une boîte personnalisée inclut la vérification de l'existence des pièces. Cela signifie que cette étape fait partie du fonctionnement normal du cas principal.

La relation extend montre qu'un cas d'utilisation peut en compléter un autre dans certaines conditions. Dans ce diagramme, elle sert à représenter des comportements optionnels ou déclenchés selon le contexte, comme la génération de la liste des pièces manquantes après l'indication qu'une boîte est incomplète.

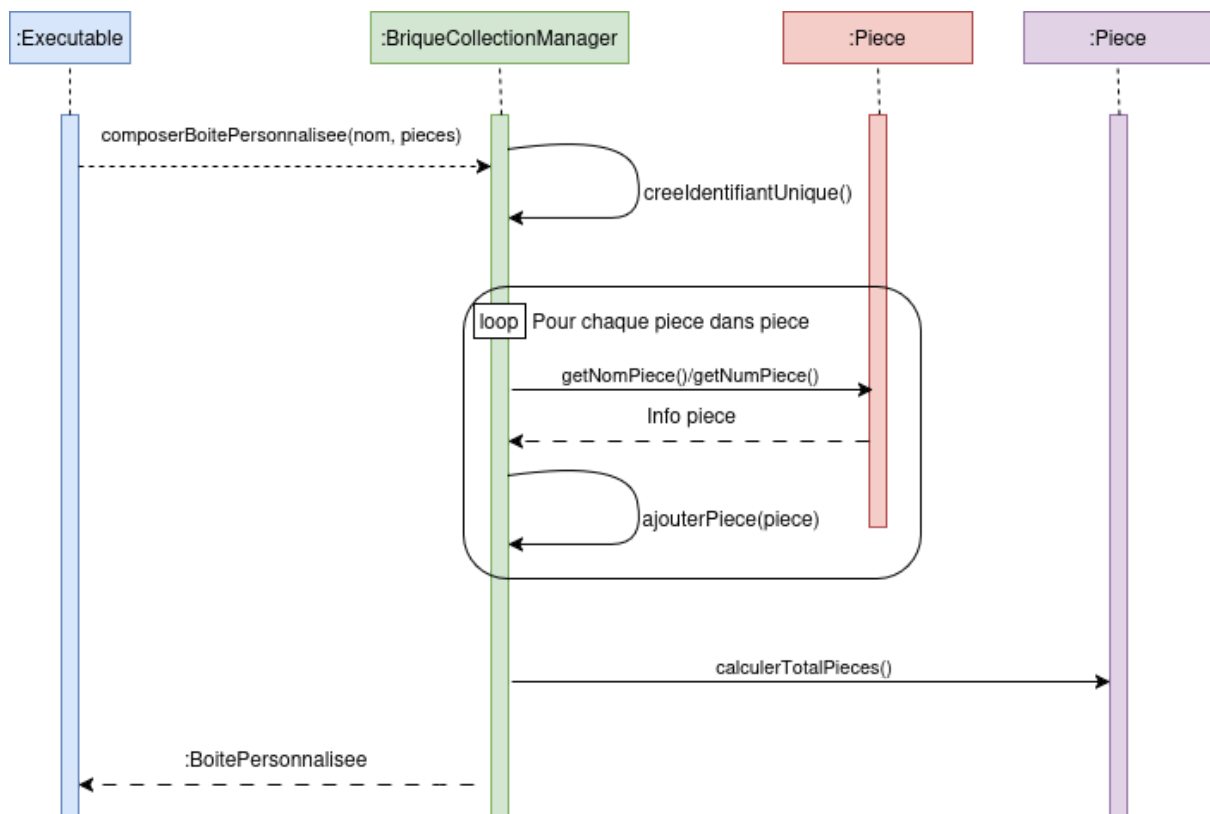
C) Présentation du diagramme de séquence

Un diagramme de séquence est un **diagramme UML (Unified Modeling Language)** qui **représente la séquence de messages entre les objets au cours d'une interaction**



Pour la recherche de boîtes par thème, notre problème était qu'un thème peut contenir des sous-thèmes, qui peuvent eux-mêmes contenir des sous-thèmes. Pour gérer ça simplement, on a ajouté une liste `sousThemes` et une méthode `estParent()` dans la classe `Theme`.

Sur le diagramme, quand l'exécutable lance `rechercherBoitesParTheme()`, le système vérifie d'abord si le thème a des enfants avec la condition `[estParent() == true]`. Si c'est le cas, il fait une boucle (`loop`) et la méthode se rappelle elle-même sur chaque sous-thème. C'est ce qu'on appelle de la récursivité. S'il n'a pas d'enfant (`[estParent() == false]`), il va chercher directement les boîtes associées à ce thème précis et les renvoie sous forme de liste (`List<Boite>`). Cette technique nous permet de fouiller automatiquement tout l'arbre des thèmes sans se soucier de sa profondeur.

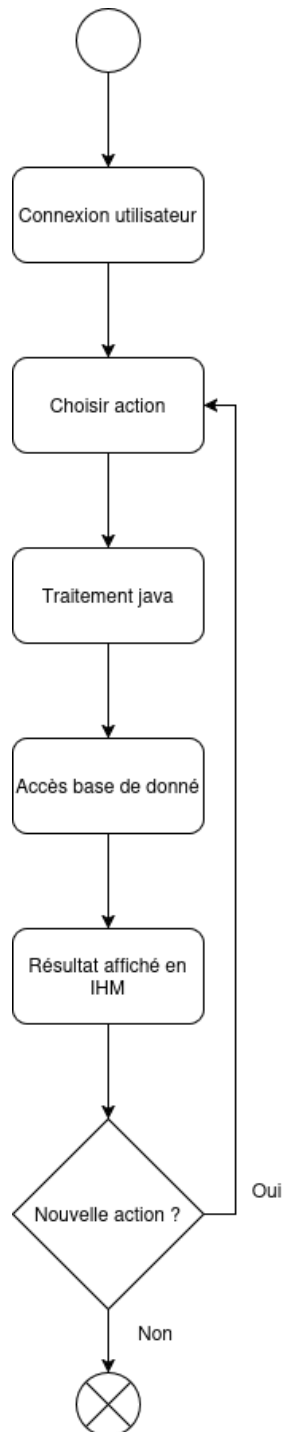


Pour la création d'une boîte personnalisée, on a voulu bien isoler la logique pour respecter le principe d'encapsulation. Au lieu de tout coder dans l'exécutable principal, on délègue le vrai travail à la classe `BoitePersonnalisee`.

L'exécutable donne simplement un nom et une liste d'objets `Piece`. Sur le diagramme, on voit que la méthode commence par appeler `creeIdentifiantUnique()` pour se générer un ID interne (grâce au `Math.random()` qu'on a mis dans le code). Ensuite, le système lance une boucle (loop) pour parcourir toutes les pièces qu'on lui a fournies. Pour chaque `Piece`, il interroge l'objet pour récupérer ses informations comme son nom ou son numéro (`getNomPiece()` et `getNumPiece()`), puis il l'ajoute à la boîte via `ajouterPiece(piece)`. Une fois que toutes les pièces sont intégrées, la méthode appelle `calculerTotalPieces()` pour mettre à jour la quantité globale, et enfin, la boîte finale est renvoyée à l'utilisateur.

Diagramme d'activité

Un diagramme d'activité **fournit une vue du comportement d'un système en décrivant la séquence d'actions d'un processus**



Ce diagramme représente le fonctionnement du système. On commence par s'identifier. Cela permet au système de proposer des actions à l'utilisateur en fonction de son rôle. L'utilisateur choisit ensuite une action à réaliser. Cette action est traitée en java. On accède

à la base de données afin de récupérer les informations nécessaires puis on affiche le résultat en IHM. Enfin, l'utilisateur décide s'il veut de nouveau effectuer une action. Si oui alors le programme recommence à partir du choix d'action. Sinon le programme s'arrête.

Implementation : explication de la démarche

Une maquette est une représentation de l'interface

A) présenter les maquettes et les choix ergonomique

The image displays three wireframe panels for a user interface, each with a title bar and a main content area.

- Page de connexion général**: Contains the text "Se connecter :", followed by input fields for "Identifiant :" and "Mot de passe :", and a "valider" button.
- Page de connexion administrateur**: Contains the text "Se connecter à un compte administrateur :", followed by input fields for "Identifiant :" and "Mot de passe :", and a "valider" button.
- Rechercher les boîtes contenant une pièce***: Contains the text "rechercher boîte par pièce", a "Numero de piece" input field, a "valider" button, and a list of sets: "Set 1", "Set 2", "Set 3", "Set 4", "Set 5", "Set 6", and "Set 7".

Cette maquette représente la page de connexion où l'utilisateur donne son identifiant et son mot de passe afin de permettre au système d'identifier le rôle de l'utilisateur ici soit collectionneur ou administrateur, le but était de faire une interface simple et neutre, le but est qu'elle soit adaptée à tout le monde est le plus rapidement compréhensible.

Ajouter une nouvelle boîte

Ajouter une nouvelle boîte

Numero de boîte

Nombre de pieces

Nom de la boîte

Année de la boîte

valider

Ajouter une nouvelle piece

Ajouter une nouvelle piece

Numero de piece

Categorie de la pieces

Nom de la piece

valider

Ajouter une nouvelle boîte

Ajouter une nouvelle boîte

Numero de boîte

Nombre de pieces

Nom de la boîte

Année de la boîte

valider

Numero de boîte deja pris

Ajouter une nouvelle piece

Ajouter une nouvelle piece

Numero de piece

Categorie de la pieces

Nom de la piece

valider

Numero de piece deja pris

Ces maquettes permettent à l'utilisateur d'ajouter une nouvelle boîte ou une nouvelle pièce. Il rentre les informations nécessaires à la création de l'objet et le système crée un nouvel objet dans la base de données. Ici des endroits permettant d'insérer du texte sont très clairement définis, le but est que l'utilisateur sache directement si oui ou non il y a un problème et qu'il est un résultat facilement accessible et compréhensible.

Créer un thème/sous-thème**

Créer un theme ou un sous theme

Numero de theme

Nom du theme

numero du theme parent (optionel)

valider

Créer un theme ou un sous theme

Numero de theme

Nom du theme

numero du theme parent (optionel)

valider

Numero de theme deja pris

Cette maquette explique comment l'administrateur crée un sous-thème. De la même manière qu'avec les maquettes précédentes, l'utilisateur rentre les informations nécessaires et le programme crée un nouveau sous-thème dans la base de données, un esthétisme simple et similaire est intégré pour faciliter la compréhension et la lisibilité de l'interface.

Mettre à jour le contenu d'une boîte**

Modifier une boîte

Numero de boîte

rechercher

Modifier une boîte

Numero de boîte

XXXXXX

Nombre de pieces

XXXXXX

Nom de la boîte

XXXXXX

Année de la boîte

XXXXXX

valider

Modifier une boîte

Numero de boîte

rechercher

Aucune boîte trouver pour ce numero

Cette maquette représente l'interface de mise à jour du contenu d'une boîte. L'administrateur rentre d'abord le numéro de la boîte. Si la boîte existe, il peut modifier les différentes informations d'une boîte, sinon un message d'erreur s'affiche. Ceci permet que l'administrateur ne fasse aucune erreur, et s' il en fait une qu'il puisse la corriger facilement.

Ajouter une boîte à sa collection

Ajouter une boîte :

Numéro de boîte

Id de collection

valider

Ajouter une boîte :

Numéro de boîte

Id de collection

valider

Numero de boîte deja pris

Ces maquette explique comment l'utilisateur peut ajouter une boîte à sa collection en rentrant son numéro de collection et son numero de boîte

Recherche de boîte

RECHERCHE DE BOÎTE LEGO®

Par numéro de boîte

Par thème et sous thème

Retour au menu précédent

RECHERCHE DE BOITE PAR NUMERO DE BOITE

Saisir le numéro de la boîte :

Résultat(s) de la recherche :

ID	Numéro	Nom	Année	Thème
...	...	...	...	...

Consulter les détails de cette boîte

Nouvelle recherche

L'ajouter à ma collection

Retour au menu principal

RECHERCHE DE BOITE PAR THEME

Saisir le thème de la boîte :

Résultat(s) de la recherche :

ID	Numéro	Nom	Année	Thème
...	...	...	...	...

Consulter les détails de cette boîte

Nouvelle recherche

L'ajouter à ma collection

Retour au menu principal

Elle explique aussi que l'utilisateur peut rechercher une boîte en tapant son numéro, son thème ou son sous-thème.

Creer une boite personnalisée

Creer une boite personnalisée

Numero de boite

Nombre de pieces

Nom de la boite

Année de la boite

valider

Ajouter une nouvelle boite

Numero de boite

Nombre de pieces

Nom de la boite

Année de la boite

valider

Numero de boite deja pris

Ajouter une nouvelle boite

valider

le numero de piece

n'existe pas

Enfin, elles expliquent que l'utilisateur peut créer une boîte personnalisée en rentrant son numéro de boîte, son nombre de pièces, le nom de la boîte personnalisé et son année.

Toutes ces interfaces sont claires, précises, le but est d'éviter au maximum l'erreur ou l'incompréhension de l'utilisateur.

Consulter le détail d'une boîte

Consulter le détail d'une boîte

Numero de boîte

Consulter le détail d'une boîte

Numero de boîte

Nombre de pièces

Nom de la boîte

Année de la boîte

Boîte complète ?

Consulter le détail d'une boîte

Aucune boîte trouvée pour ce numéro

Liste des pièces manquantes

L'utilisateur peut consulter le détail d'une boîte en rentrant son numéro de boîte et avoir ainsi accès au numéro de boîte, le nombre de pièce dans la boîte, le nom de la boîte, l'année de la boîte et vérifier si elle est complète. Si le numéro de boîte donné par l'utilisateur n'existe pas, un message d'erreur apparaît permettant à l'utilisateur de modifier le numéro de boîte, et donc de corriger l'erreur facilement.

Elaboration des Interfaces Hommes Machine

A) Expliquez la démarche (développement du back-end : traitements, calculs)

Organisation du travail de groupe

A) Comment vous êtes-vous organisés tout au long de cette SAE ?

Au début de chaque séance, une revue des tâches déjà réalisées est effectuée. À la suite de cette revue, le chef de groupe identifie les objectifs à accomplir durant la séance.

Ensuite, le chef de projet répartit les tâches en fonction des compétences de chacun. Certaines tâches peuvent être attribuées à deux personnes afin de réduire leur temps d'exécution et de permettre au groupe de se concentrer sur les éléments les plus importants.

Chaque membre du groupe travaille alors sur les missions qui lui sont confiées. Une fois une tâche terminée, une nouvelle lui est attribuée.

Un dépôt GitHub a été mis en place dès la première séance afin de simplifier le partage des tâches ainsi que le suivi de leur réalisation. Un document partagé Google Docs a également été créé dans le même objectif.

B) Présentez vos outils de gestion de projet (Diagramme de Gantt, matrice RACI)

		ADIL	ZAKARIA	FLORIENT	NATHANIEL
ID	Livrable ou tâche				Chef de projet
1	Gestion de Projet	R	I	R	A
2	Analyse & Conception	R	I	I	A
3	Développement Java	I	R	R	A
4	Qualité & Tests	I	R	R	A
5	Implémentation	R	R	I	A
6	Livrables Écrits	R	R	R	A
7	Soutenance	R	R	R	A

Pour s'organiser sur ce projet et éviter de tous faire la même chose en même temps, on a utilisé une matrice RACI. L'idée, c'était de savoir exactement qui était responsable de quoi.

Le pilotage : Déjà, Nathaniel est notre chef de projet. Dans le tableau, il a le rôle "A" (Approbateur) sur absolument toutes les lignes. Son rôle officiel, c'était de donner le feu vert à chaque étape. Mais attention, ça ne veut pas dire qu'il n'a rien fait ! Sur notre tableau de suivi, on voit bien que c'est lui qui a mis en place notre dépôt Git, géré les branches et corrigé les documents.

L'Analyse et la Conception (UML) : Officiellement sur le RACI, c'est Adil qui a le "R" de réalisateur pour cette partie. Il s'est occupé des explications détaillées des diagrammes et de la base de données. Mais en pratique, on a travaillé en équipe pour avancer plus vite : par exemple, Zakaria s'est occupé de traduire le diagramme des cas d'utilisation et Florient a fait le diagramme d'activité.

Le Code et les Tests : Pour tout ce qui est développement Java et tests, on a confié les rôles de réalisateurs ("R") à Zakaria et Florient. Pendant ce temps, Adil était simplement tenu informé ("I") pour savoir où le code en était.

Le Travail Collaboratif : Enfin, pour la gestion de projet et la rédaction du rapport (les livrables écrits), on s'est vraiment partagé le boulot, c'est pour ça qu'il y a plusieurs "R". Par exemple, pour la gestion, Florient a créé le diagramme de Gantt et Adil a fait cette matrice RACI. Pour le rapport, on s'est divisé les parties : Florient a géré la mise en page et l'explication des modules, pendant que Zakaria et Adil rédigeaient les parties plus techniques sur le MCD et l'UML.

En gros, la matrice nous a donné un super cadre de départ, et ensuite, on s'estentraîdés intelligemment pendant les séances pour tout boucler dans les temps

	Semaine d'affichage :	1	Séance 1	Séance 2	Séance 3	Séance 4	Séance 5	Séance 6	Séance 7	Séance 8
TÂCHE										
Prise de connaissance du sujet										
création du document partagé										
création de la page de garde										
créations de branches git										
mise en place du dépôt git										
Réalisation de la page de garde										
mise forme du documents										
Création / actualisation de la table de matière										
explication des module										
explication des ressources mobilisés et combinés										
explication des apprentissages critiques										
correction de l'introduction										
Mise en place du projet et rédaction de l'introduction du rapport										
Réalisation du MCD										
traduction en MD et rédaction de leurs explications détaillées										
Approfondissement sur l'explication des modules.										
réalisation des deux diagrammes										
mise à jour du GitHub										
mise à jour du google Docs										
réalisation de la présentation des instances										
mise à jour du google Docs										
réalisation de la présentation des requêtes										
mise à jour de la table des matières										
traduction du diagramme de cas d'utilisation										
Finalisation de la traduction du MCD en MD										
correction de la modélisation										
rédaction finale de l'explication détaillée de la base de données pour le rapport										
Création du diagramme de Gant										
Création du diagramme de Gant										
Création du diagramme d'activité										
Mise à jour des diagrammes de séquence										
mise a jour du diagramme UML										
correction des multiples problèmes lié au GitHub et au Google Docs										
Mise à jour du diagramme de classe										
Création de la matrice RACI avec début d'explication et brouillon pour les explications des diagrammes										
Création du Gantt.										
Création des explications des travaux réalisés										
Correction de la mise en page du Google Docs										
mise à jour de la table des matières										
Mise à jour du diagramme de classe										
Rendu au propre des brouillon des explications										
mise à jour des explications de la matrice RACI										
Réalisation des maquettes										
Correction des problèmes GitHub										
ajout du diagramme de classe sur le Google Docs										
réalisation des dernières maquettes										
ajout de l'explication ergonomiques des maquettes										
réorganisation des onglets du document										
modification de la table des matières										

Explication du Gantt

Ceci est notre diagramme de Gantt qui représente par l'en toute les tache accompli à chaque séance. De cette manière chaque tâche était réalisée pendant une séance unique, cependant quelque tâche devait-être retravaillée, notamment les diagrammes étant couramment modifier en fonction des besoins de la SAE, en conséquence, les explications était alors modifier. De plus, les maquettes ont pris deux séances, en raison du grand nombre de maquettes à réaliser. L'organisation du document était notamment modifiée et mise a jour a chaque séance en fonction des différents document et texte ajouter.

C) Présentez GitHub et son utilité et expliquez la manière dont vous l'avez utilisé

GitHub a été un outil clé durant notre SAE. En effet, l'ensemble du travail de chaque membre du groupe était centralisé sur notre dépôt GitHub. Son objectif était de simplifier le travail collaboratif, puisque chacun pouvait récupérer, modifier et partager les fichiers du projet facilement.

Pour chaque fonction ou classe développée, ainsi que pour les tests qui y étaient associés, un commit était réalisé. Un commit consiste à enregistrer les modifications effectuées sur le projet avec un message précis décrivant les changements apportés. Cette méthode permettait de justifier et de détailler un maximum d'informations, afin de simplifier la communication entre les membres du groupe et de gagner du temps.

En conclusion, GitHub a été l'outil le plus utilisé durant notre SAE. Il nous a permis de gagner un temps considérable et a fortement contribué à la rapidité ainsi qu'à la simplification des tâches.

Travail réalisé durant les séances

Séance n°	Florient	Nathaniel	Zakaria	Adil	Mourad
1	Prise de connaissance du sujet, création du document partagé, création de la page de garde, créations de branches git.	Prise de connaissance du sujet, création du document partagé, mise en place du dépôt git, créations de branches git.	Prise de connaissance du sujet, création du document partagé, mise en place du dépôt git, créations de branches git.	Prise de connaissance du sujet, création du document partagé, mise en place du dépôt git, créations de branches git.	pas encore rejoint leur groupe
2	Réalisation de la page de garde, mise forme du documents, création de la table de matière, explication des module, explication des ressources mobilisés et combinés	Réalisation de la page de garde, mise forme du documents, création de la table de matière, explication des module, explication des apprentissages critiques, correction de l'introduction	Mise en place du projet et rédaction de l'introduction du rapport. Réalisation du MCD, traduction en MLD et rédaction de leurs explications détaillées. Approfondissement sur l'explication des modules.	Mise en place du projet et rédaction de l'introduction du rapport. Réalisation du MCD, traduction en MLD et rédaction de leurs explications détaillées	pas encore rejoint leur groupe

3	réalisation des deux diagrammes, mise à jour du GitHub, mise à jour du google Docs, réalisation de la présentation des instances	réalisation des deux diagrammes, mise à jour du google Docs, réalisation de la présentation des requêtes, mise à jour de la table des matières	analyse de l'UML : traduction du diagramme de cas d'utilisation	Finalisation de la traduction du MCD en MLD, correction de la modélisation et rédaction finale de l'explication détaillée de la base de données pour le rapport	pas encore rejoint leur groupe
4	Création du diagramme de Gantt. Création du diagramme d'activité	Mise à jour des diagrammes de séquence, mise à jour du diagramme UML, correction des multiples problèmes liés au Github et au Google Docs	Mise à jour du diagramme de classe	Création de la matrice RACI avec début d'explication et brouillon pour les explications des diagrammes de séquence et du diagramme UML	Prise de connaissance du sujet, création du document partagé, mise en place du dépôt git, créations de branches git.
5	Création du Gantt.	Création des explications des travaux réalisés, de la manière de travailler, de l'utilisation du GitHub durant la SAE.	Mise à jour du diagramme de classe	Rendu au propre des brouillons des explications des diagrammes de séquence et amélioration du brouillon	mise à jour du GitHub, mise à jour du google Docs, aide à la préparation du Gantt

		Correction de la mise en page du Google Docs, mise à jour de la table des matières		du diagramme UML, et mise à jour des explications de la matrice RACI	
6	Réalisation des maquettes : - Ajouter une boîte à sa collection - Rechercher les boîtes contenant une pièce - Ajouter une nouvelle boîte - Ajouter une nouvelle pièce - Créer un thème	Réalisation des maquettes, Correction des problèmes GitHub, ajout du diagramme de classe sur le Google Docs	Correction des analyses des différents diagrammes, mise à jour des erreurs observées sur les diagrammes	Réalisation des maquettes	Réalisation des maquettes, mise à jour des erreurs observées sur les diagrammes

	e/so us-th ème				
	Mettre à jour le contenu d'une boîte**				
	Créer une boîte personnalis ée				
7	réalisation des dernières maquettes, ajout de l'explication ergonomiqu es des maquettes, réorganisati on des onglets du document, modification de la table des matières	réalisation des dernières maquettes, ajout de l'explication ergonomiqu es des maquettes, réorganisati on des onglets du document, modification de la table des matières	réalisation des dernières maquettes, ajout de l'explicatio n ergonomiq ues des maquettes, réorganisa tion des onglets du document, modificatio n de la table des matières	Absent	réalisation des dernières maquettes, ajout de l'explicatio n ergonomiq ues des maquettes,
8				Avanceme nt du diaporama	aide a la finalisation du diaporama

Bilan du groupe

A) Qu'est-ce qui fonctionne, qu'est-ce qui ne fonctionne pas, et pourquoi ?

Le groupe n'a pas rencontré de problème d'organisation, le travail était correctement réparti au début de chaque séance, il était réparti en fonction des avantages de chacun, le travail étant réalisé par plusieurs personnes en même temps permettait de terminer les tâches plus rapidement, et une relecture était réalisée pour éviter le maximum d'erreur, une bonne communication globale était présente dans le groupe, ceci ne freinait jamais le travail à faire.

B) Bilan sur l'organisation du groupe et du projet

L'organisation du groupe était faite à chaque séance par le chef de projet en fonction des compétences de chacun, ceci aura permis un grand gains de temps durant tous le projet, le projet était organisée en plusieurs section, la partie rapport et la partie code, la partie rapport était plus dirigée sur la plateforme Google Docs, et la partie code étant plus dirigée sur la plateforme github.

C) Bilan sur les principaux acquis

Nous avons tous amélioré nos compétences de communication, à réaliser des tâches avec les différents membres du groupe, ceci grâce au différent plateforme d'échange mise en place (Google Docs et github). De manière générale, l'utilisation et l'optimisation du temps durant chaque séance a été grandement améliorée, permettant de compléter un grand nombre de tâches pendant une unique séance. Les compétences de rédaction de chacun ont notamment évolué, durant le projet de nombreuses explications, rendu ou rapport devait-être réalisé. L'organisation globale du groupe a notamment été améliorée, les tâches réparties entre chaque personne et des tâches clairement définies au début de la séance permettra donc un grand gain de temps, le google docs étant l'outil majeure du projet était nécessairement bien organisé, avec des onglets de document et une table de matière claire et précise.